



Saving Data.

Day 8 you built an app. Today we make its data survive — JSON for config, SQLite for real storage. The skills that turn a demo into a product.

PROGRAM

Python Internship

LED BY

Zlaark

SESSION

9 of 24

How we'll spend our 90 minutes today.

Two storage formats, one real database, one Streamlit pattern, and an exercise that upgrades Day 8's expense tracker from CSV to SQLite.

DURATION

90 min

00:00 - 00:10



Day 8 Recap & Q&A

The solo challenge, the expense tracker, what worked and what broke.

00:10 - 00:25



JSON — dicts to files

Python's `json` module. `json.dump`, `json.load`. Good for config and nested data.

00:25 - 00:50



SQLite — a real database

Python's `sqlite3` module. `CREATE TABLE`, `INSERT`, `SELECT`. The storage every production app uses.

00:50 - 01:00



SQLite in Streamlit

The connection-per-widget pattern. Streamlit re-runs everything — so you open and close the connection each time.

01:00 - 01:10



Storage Cheatsheet

JSON vs CSV vs SQLite. When to use which. A printable reference.

01:10 - 01:30



Exercise: Expense Tracker with SQLite

Rebuild Day 8's app with a real database. `CREATE TABLE`, `INSERT` on submit, `SELECT` to display.

Yesterday you built alone. Today we make it persist.

Quick check that Day 8's expense tracker exists. If it doesn't, we start from a scaffold — today's exercise upgrades it to SQLite.

WHAT YOU DID YESTERDAY DAY 08

- ✓ **Built a personal expense tracker** — solo, 60 minutes
- ✓ **Added expenses** through a Streamlit form
- ✓ **Stored them in a CSV file** (`pandas read_csv` / `to_csv`)
- ✓ **Showed the total**, a chart, and a table
- ✓ **Demoed (or watched demos)** — the gate before group formation

CHECK YOUR LAPTOP NOW VERIFY

```
# You should have Day 8's app:
$ ls day8/
app.py
expenses.csv

# Run it — should still work:
$ streamlit run app.py
# → Browser opens, form works
# → Expenses persist between runs
```

⚠ **IF YOUR APP IS MISSING** — pair up with a neighbour. Today's exercise rebuilds it with SQLite, so you need the original to compare. We'll provide a scaffold if needed.

Your app needs to remember. Pick the right storage.

Day 8 used CSV — simple, flat, fine for 50 rows. Today we add JSON (for nested data) and SQLite (for real database operations). Different tools for different jobs.

CSV



Flat tables. Rows and columns. What you used on Day 8.

✓ GOOD FOR

Simple tabular data (expenses, student records, logs). Easy to inspect in a text editor.

✗ NOT FOR

Nested data, relationships between tables, concurrent writes.

JSON



Nested dicts and lists. Python's `json` module.

✓ GOOD FOR

Config files, settings, data with structure (a user profile with nested address). API responses.

✗ NOT FOR

Large datasets (10,000+ rows), querying/filtering, multiple writers.

SQLite



A real database in a single file. Python's `sqlite3` module.

✓ GOOD FOR

Anything with relationships, queries, filtering, or more than a few hundred rows. The storage every production app uses.

✗ NOT FOR

Simple config — JSON is easier. Read-only data that never changes.

KEY IDEA CSV is a flat file. JSON is a nested file. SQLite is a database in a file. They are not interchangeable — pick based on your data's shape and how you'll query it.

JSON — save Python dicts to a file. Load them back.

JSON (JavaScript Object Notation) is a text format for nested data. Python's `json` module converts dicts/lists to JSON strings and back. Good for config, settings, and any data that isn't a flat table.

```
json_demo.py PYTHON

import json

# === Save a dict to a file ===
config = {
    "app_name": "Expense Tracker",
    "version": "1.0",
    "categories": ["Food", "Transport", "Books"],
    "settings": {
        "currency": "INR",
        "dark_mode": False
    }
}

with open("config.json", "w") as f:
    json.dump(config, f, indent=4)
# → writes a nicely-formatted JSON file

# === Load it back ===
with open("config.json", "r") as f:
    loaded = json.load(f)

print(loaded["app_name"])    # → Expense Tracker
print(loaded["categories"])  # → ['Food', 'Transport', 'Books']
print(loaded["settings"]["currency"]) # → INR
```

1 `json.dump(obj, file)` — Writes a Python object (dict/list) to a file as JSON. Use `indent=4` for pretty-printing.

USE WHEN Your data is nested or structured (not a flat table), and you need to read/write the whole thing at once. Avoid for large datasets — JSON loads the entire file into memory.

WHEN TO USE JSON DECIDE



Config files

App settings, user preferences, feature flags. Read once on startup, written on change.



Nested data

A user profile with an address, a list of orders, each with items. JSON handles nesting naturally; CSV doesn't.



API responses

Most web APIs return JSON. Day 11 covers this — you'll use `json.loads()` to parse responses.



Small + read-mostly

Data that changes rarely and is read often. JSON files are simple to inspect and edit by hand.

A database is a **structured way to store data** so you can find it again.

You've been storing data in CSV files — flat tables. A database is the same idea, but structured: tables with named columns, rules about what goes in them, and a query language (SQL) to ask questions of the data.

THE SPREADSHEET YOU ALREADY KNOW

If you've used Excel or Google Sheets, you already understand databases. The mapping is one-to-one:



Spreadsheet → Database. One Excel file is one database. It holds all your data in one place.



Worksheet → Table. Each tab at the bottom of Excel is a table. A database can have many tables.



Column → Column. Named, typed (text, number, date). `name`, `age`, `marks`.



Row → Row. One record. One student, one expense, one order. Each row has a value for every column.



Filter/sort → SQL query. Instead of clicking buttons, you write `SELECT * FROM students WHERE age > 20`.

A DATABASE TABLE — STUDENTS

id	name	age	city
1	Aarav	20	Amritsar
2	Priya	21	Jalandhar
3	Rahul	22	Amritsar
4	Meera	20	Ludhiana

Looks like a spreadsheet. But behind it: a query engine, types, rules, and persistence.

THE DIFFERENCE

A CSV file is just **text on disk** — you load it all into memory to use it. A database is **software that manages the data** — it indexes it, enforces rules, handles concurrent writes, and lets you query without loading everything.

KEY IDEA A database is a spreadsheet with superpowers. Same mental model (tables, rows, columns), but it scales to millions of rows, supports relationships between tables, and answers questions in milliseconds. **SQL is the language you use to talk to it.**

Tables, rows, columns, and data types.

The structure of every database. Once you can name these parts, the SQL syntax becomes obvious.

THE FOUR PARTS

- 1 Table** — A named collection of rows. Like one worksheet in Excel. A database has many tables: `students`, `expenses`, `courses`.
- 2 Column** — A named field in a table, with a type. `name (TEXT)`, `age (INTEGER)`, `marks (REAL)`. Defined when the table is created.
- 3 Row** — One record. One student, one expense. Has a value for every column. The unit you INSERT, SELECT, UPDATE, DELETE.
- 4 Primary Key** — A unique ID for each row. Usually an auto-incrementing integer (`id = 1, 2, 3...`). How you find a specific row.

COMMON DATA TYPES (SQLITE)

INTEGER

Whole numbers. Age, count, id. `20`, `450`, `1`

REAL

Decimal numbers. Amounts, prices. `450.50`, `3.14`

TEXT

Strings. Names, categories, notes. `"Aarav"`, `"Food"`

BLOB

Binary data. Images, files. Rare in beginner apps.

NULL

No value. Not the same as 0 or `""`. Means "unknown" or "not set".

DATETIME

Dates and times. SQLite stores as TEXT (ISO format). `"2026-08-05"`

SQL – the 4 operations

SQL

```
-- Create a table
CREATE TABLE students (
  id INTEGER PRIMARY KEY,
  name TEXT,
  age INTEGER
);

-- Add a row
INSERT INTO students (name, age) VALUES ('Aarav', 20);

-- Read rows
SELECT * FROM students WHERE age > 18;
```

KEY IDEA Every database operation is one of four verbs: **CREATE** (make a table), **INSERT** (add a row), **SELECT** (read rows), **UPDATE/DELETE** (modify rows). That's it. The rest is just syntax and types.

CSV works for 50 rows. Then **it breaks**.

A CSV file is just text. A database is software that manages data. Here are the four problems databases solve that CSV can't.



1. Querying without loading everything

To find "all expenses over ₹500 in a CSV, you load the **entire file** into pandas, then filter. With 10,000 rows, that's slow. A database uses **indexes** to find matching rows without scanning the whole table.

```
-- Database: finds in milliseconds
SELECT * FROM expenses WHERE amount > 500;
```



2. Relationships between tables

A student has many expenses. An expense belongs to one category. In CSV, you'd duplicate the student name on every row. In a database, you use **foreign keys** — one `students` table, one `expenses` table, linked by `student_id`.

```
students: id=1, name="Aarav"
expenses: student_id=1, amount=450
```



3. Concurrent writes (multiple users)

Two users add an expense at the same time to a CSV? **Last write wins, the first is lost**. A database handles concurrent writes with locking and transactions — both saves succeed, no data lost.

```
-- Both run at the same time, both succeed
INSERT INTO expenses ...;
INSERT INTO expenses ...;
```



4. Data integrity and rules

A CSV will happily let you put "abc" in the amount column. A database enforces **types and constraints** — `amount REAL` rejects text, `NOT NULL` rejects empty, `UNIQUE` rejects duplicates. Bad data is caught at insert, not at runtime.

```
amount REAL NOT NULL
-- INSERT ... amount='abc' → ERROR
```

WHEN TO UPGRADE Start with CSV (Day 8's app). When you hit **any** of these four problems — queries are slow, you need relationships, multiple users write at once, or bad data keeps sneaking in — upgrade to SQLite. Today's exercise does exactly that.

Five databases. Same SQL. Different jobs.

They all speak SQL. The difference is where they run, how much they handle, and who maintains them. We use SQLite today — but you'll meet the others in every job after college.

SQLite

EMBEDDED • FILE-BASED

A database **in a single file**. No server. Python's `sqlite3` module is built in. Used by every phone, browser, and desktop app.

GOOD FOR Beginners, apps, small data

SERVER? No — it's a file

SCALE ~100k rows, single user

MySQL

CLIENT-SERVER • OPEN SOURCE

A **server you connect to**. Runs separately from your app. Used by WordPress, Facebook, Twitter. The most common database on the web.

GOOD FOR Web apps, multi-user

SERVER? Yes — runs as a service

SCALE Millions of rows, many users

PostgreSQL

CLIENT-SERVER • OPEN SOURCE

Like MySQL, but **more powerful**. Handles complex queries, JSON columns, full-text search. The choice for data-heavy apps and startups.

GOOD FOR Complex apps, analytics

SERVER? Yes — runs as a service

SCALE Billions of rows, advanced

MariaDB

CLIENT-SERVER • OPEN SOURCE

A **fork of MySQL** by the original developers. Same syntax, compatible drop-in replacement. Used where MySQL's Oracle ownership is a concern.

GOOD FOR Same as MySQL

SERVER? Yes — runs as a service

SCALE Same as MySQL

OracleDB

CLIENT-SERVER • ENTERPRISE

The **enterprise giant**. Banks, governments, airlines. Expensive, powerful, complex. You won't use it in this program, but you'll see it in job descriptions.

GOOD FOR Banks, enterprises, legacy

SERVER? Yes — dedicated hardware

SCALE Massive, mission-critical

Today: SQLite.

Same SQL works everywhere.

Learn SQLite today → switch to MySQL/PostgreSQL later by changing one connection string.

THE PATTERN All five speak **SQL**. Learn `SELECT`, `INSERT`, `UPDATE`, `DELETE` on SQLite today — the same four commands work on every database in the industry.

DAY 09 • SAVING DATA

09 / 25

Same SQL. Different strengths. Different costs.

Data types are one dimension. Here are eight more — the real differences that matter when picking a database for a project.

DIMENSION	SQLITE EMBEDDED	WE USE THIS	MYSQL CLIENT-SERVER	POSTGRES CLIENT-SERVER	MARIADB CLIENT-SERVER	ORACLEDB ENTERPRISE
Architecture	Embedded (in-process, file-based)		Client-server	Client-server	Client-server	Client-server (enterprise)
Setup	Zero config — just a file		Install + configure server	Install + configure server	Same as MySQL	Complex — DBA recommended
Licensing	Public domain (free)		GPL / Commercial dual-license	PostgreSQL License (free, BSD-like)	GPL / LGPL (free)	Proprietary (\$\$ — thousands/year)
Concurrency	Single writer, multiple readers		Multi-writer with row-level locking	MVCC (multi-version concurrency)	Same as MySQL	Advanced — row + table-level
Scalability	~100K rows, single machine		Millions of rows, read replicas	Billions of rows, advanced partitioning	Same as MySQL	Massive — clustering, sharding
ACID	Yes (simplified)		Yes (full)	Yes (full, advanced isolation)	Yes (full)	Yes (enterprise, distributed)
Replication	None (single file)		Master-slave, master-master	Streaming + logical	Master-slave, galera cluster	Data Guard, RAC
Stored Procedures	Limited (SQL functions)		Yes (SQL + procedures)	Yes (PL/pgSQL — powerful)	Yes (SQL + procedures)	Yes (PL/SQL — most advanced)
Real-world Users	Every phone, browser, desktop app		WordPress, Facebook, Twitter (early)	Instagram, Spotify, Reddit	Wikipedia, Google (some)	Banks, airlines, governments

THE BIG DIFFERENCE SQLite is a **FILE**. The others are **SERVERS**. That's why SQLite has no replication, no concurrency, and zero setup — it's not a server, it's a library. The others are software that runs as a service.

Which database for **which** job?

Five scenarios. Five picks. The decision tree you'll use in every project, from a college assignment to an enterprise system.

1

Personal app, single user, learning

Building an expense tracker, a note app, a personal dashboard. **One user**. Data on one laptop.

PICK

 SQLite

WHY: Zero setup. No server. Just a file. You're already using it today. Perfect for prototypes and personal tools.

2

Web app, multiple users, growing data

A blog, an e-commerce site, a SaaS product. **Hundreds of users writing at once**. Need it online 24/7.

PICK

 MySQL or PostgreSQL

WHY: Client-server architecture. Handles concurrent writes. Read replicas for scaling. Free and battle-tested. WordPress uses MySQL; Instagram uses PostgreSQL.

3

Complex data, analytics, JSON columns

An app with nested data (**JSON in columns**), complex queries, full-text search, geospatial data.

PICK

 PostgreSQL

WHY: The most powerful open-source database. JSON columns (JSONB), full-text search, PostGIS for maps, advanced indexing. The choice for data-heavy apps and startups.

4

MySQL compatibility, no Oracle ownership

You want MySQL's features but prefer a **fully open-source fork** maintained by the original developers.

PICK

 MariaDB

WHY: Drop-in replacement for MySQL. Same syntax, same connectors. Used by Wikipedia. The political choice — no Oracle licensing concerns.

5

Enterprise, mission-critical, budget available

A bank, an airline booking system, a government database. **Millions of transactions. Zero downtime**. Budget for licensing and DBAs.

PICK

 OracleDB

WHY: Enterprise-grade. Clustering (RAC), data guard, advanced security. Expensive — thousands per year per CPU. You won't use it here, but you'll see it in enterprise job descriptions.

THE RULE Start with **SQLite** (today). Upgrade to **PostgreSQL** when you outgrow it (Phase 4 group project, if needed). MySQL/MariaDB for web apps. OracleDB for enterprise jobs. The SQL you learn today transfers to all five.

Two families. Different shapes of data.

All five databases from the last slide are **relational (SQL)**. But there's a whole other family — **non-relational (NoSQL)** — for data that doesn't fit in tables. Know both exist; know when to pick which.



Relational (SQL)

STRUCTURED • TABLES • STRICT SCHEMA

SHAPE Data lives in **tables** with fixed columns. Every row has the same structure. Like a spreadsheet.

SCHEMA **Fixed.** You define columns and types upfront. Adding a column later is a migration.

QUERY `SELECT * FROM students WHERE age > 20` — SQL, the universal language.

RELATIONS **Foreign keys** link tables. Students → expenses via `student_id`.

BEST FOR Structured data, relationships, transactions, complex queries. Most business apps.

SQLite

MySQL

PostgreSQL

MariaDB

OracleDB



Non-Relational (NoSQL)

FLEXIBLE • DOCUMENTS • GRAPH • KEY-VALUE

SHAPE Data lives as **documents** (JSON-like), key-value pairs, or graphs. Each record can have different fields.

SCHEMA **Flexible.** Add a new field to one record without changing the others. No migrations.

QUERY Database-specific. MongoDB: `db.users.find({"age": {"$gt": 20}})`. No universal language.

RELATIONS **Embedded or referenced.** A user document can contain their orders nested inside. Or reference by ID.

BEST FOR Unstructured data, rapid prototyping, big data, content management, real-time apps.

MongoDB

Redis

Cassandra

Firebase

DynamoDB

WHICH ONE? Start with **SQL (SQLite)** — it's what we use today and what 80% of jobs use. Learn NoSQL when you hit data that doesn't fit in tables (nested, changing structure, massive scale). Both are valuable; they solve different problems.

Same idea, different names.

Every database needs to store text, whole numbers, decimals, dates, and booleans. The types are the same — the names differ slightly. Here's the cheat-sheet.

CONCEPT	SQLITE	MYSQL	POSTGRESQL	MARIADB	ORACLEDB
Whole number	INTEGER	INT	INTEGER	INT	NUMBER(10)
Small number	INTEGER	TINYINT	SMALLINT	TINYINT	NUMBER(3)
Decimal	REAL	FLOAT / DOUBLE	REAL / NUMERIC	FLOAT / DOUBLE	FLOAT / NUMBER(p, s)
Money	REAL	DECIMAL(10, 2)	NUMERIC(10, 2)	DECIMAL(10, 2)	NUMBER(10, 2)
Short text	TEXT	VARCHAR(255)	VARCHAR(255)	VARCHAR(255)	VARCHAR2(255)
Long text	TEXT	TEXT / LONGTEXT	TEXT	TEXT / LONGTEXT	CLOB
Date	TEXT (ISO)	DATE	DATE	DATE	DATE
Date + time	TEXT (ISO)	DATETIME	TIMESTAMP	DATETIME	TIMESTAMP
Boolean	INTEGER (0/1)	BOOLEAN / TINYINT	BOOLEAN	BOOLEAN / TINYINT	NUMBER(1)
Binary (files)	BLOB	BLOB / LONGBLOB	BYTEA	BLOB / LONGBLOB	BLOB
JSON	TEXT	JSON	JSONB	JSON / LONGTEXT	CLOB / JSON
Auto-increment ID	INTEGER PRIMARY KEY	AUTO_INCREMENT	SERIAL	AUTO_INCREMENT	GENERATED ALWAYS AS IDENTITY

THE PATTERN Learn SQLite's types today (`INTEGER` , `REAL` , `TEXT` , `BLOB`). When you switch to MySQL/PostgreSQL, the names change slightly but the concepts are identical — you'll pick them up in an afternoon. **The hard part is SQL, not the type names.**

Four categories. One language.

SQL isn't one thing — it's four categories of statements, each with a job. You'll use DQL and DML every day. DDL when you set up the database. DCL rarely.

DDL Data Definition Language

Define the structure. Create tables, change columns, set rules. Run once when setting up the database.

CREATE TABLE students (id INTEGER, name TEXT)

ALTER TABLE students ADD COLUMN email TEXT

DROP TABLE students — deletes the whole table

TRUNCATE students — empties but keeps structure

DML Data Manipulation Language

Change the data. Add, update, delete rows. The commands you'll use every day in your app.

INSERT INTO students VALUES (1, 'Aarav')

UPDATE students SET age = 21 WHERE id = 1

DELETE FROM students WHERE id = 1

DQL Data Query Language

Read the data. Ask questions. Filter, sort, aggregate. The most-used category by far.

SELECT * FROM students

SELECT * FROM students WHERE age > 20

SELECT city, COUNT(*) FROM students GROUP BY city

SELECT * FROM students ORDER BY marks DESC LIMIT 10

DCL Data Control Language

Control access. Who can do what. Used by DBAs, not app developers. You'll rarely write these.

GRANT SELECT ON students TO user_aarav

REVOKE DELETE ON students FROM user_aarav

BEGIN TRANSACTION — start a transaction

COMMIT / **ROLLBACK** — save / undo

TODAY You'll use **CREATE TABLE** (DDL) once, then **INSERT** / **UPDATE** / **DELETE** (DML) and **SELECT** (DQL) all day. That's 90% of real database work. **DCL is for DBAs — you'll meet it in jobs, not in this program.**

Your first SQL query. Three words, one result.

Before we dive into CREATE TABLE and INSERT, let's see what a query actually looks like. This is the pattern — you write SQL, the database returns rows.

```
first_query.py PYTHON

import sqlite3

# 1. Connect to the database
conn = sqlite3.connect("students.db")
cursor = conn.cursor()

# 2. Write a SQL query
cursor.execute("SELECT * FROM students")

# 3. Fetch the results
rows = cursor.fetchall()

# 4. Print them
for row in rows:
    print(row)

# 5. Close
conn.close()
```

- 1 `sqlite3.connect()` — Opens the database file. Creates it if missing.
- 2 `cursor.execute(SQL)` — Runs your SQL. The SQL is a string.
- 3 `fetchall()` — Returns a list of tuples. Each tuple is a row.

WHAT YOU SEE OUTPUT

```
$ python first_query.py
(1, 'Aarav', 20, 'Amritsar', 'BCA', 85)
(2, 'Priya', 21, 'Jalandhar', 'BCA', 92)
(3, 'Rahul', 22, 'Amritsar', 'BSc-IT', 78)
(4, 'Meera', 20, 'Ludhiana', 'BCA', 88)
```

Each tuple is a row. `SELECT *` returns every column. The order matches the table schema.

KEY IDEA

You write SQL as a string, pass it to `execute()`, get rows back as tuples. That's the whole pattern. **Everything else is variations.**

A cursor is your **handle** for running SQL.

You've seen `cursor = conn.cursor()` and `cursor.execute()` on the last slide. But what IS a cursor? **Think of it as a remote control** — you use it to send SQL commands and receive results.

```

● ● ● cursor_explained.py PYTHON

import sqlite3

# Step 1: Connect to the database
conn = sqlite3.connect("students.db")
# conn is the CONNECTION — the pipe to the file

# Step 2: Create a cursor
cursor = conn.cursor()
# cursor is the REMOTE CONTROL — you use it to run SQL

# Step 3: Run a query through the cursor
cursor.execute("SELECT * FROM students")
# The cursor sends the SQL to the database,
# and holds the results (but doesn't show them yet)

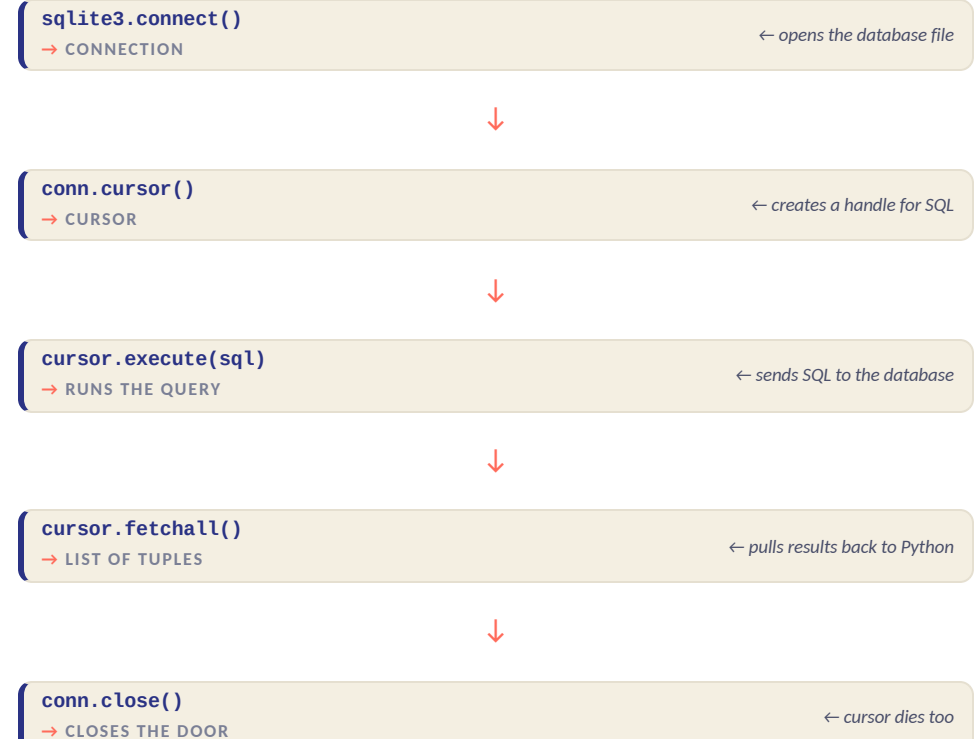
# Step 4: Fetch the results
rows = cursor.fetchall()
# NOW the results come back to Python as a list of tuples

# Step 5: Close the connection
conn.close()
# The cursor dies with the connection. No cursor without a conn.

```

- 1 `conn = sqlite3.connect()` — The CONNECTION. Opens the database file. Like opening a door to the database.
- 2 `cursor = conn.cursor()` — The CURSOR. Creates a 'remote control' for sending SQL through the connection. One connection can have many cursors.
- 3 `cursor.execute(sql) + cursor.fetchall()` — Send SQL, get results. `execute()` runs the query; `fetchall()` pulls the rows back to Python.

THE FLOW CONNECTION → CURSOR → RESULTS



ANALOGY

Connection = a phone call to the database. Cursor = the person speaking. You talk through the cursor. When you hang up (`close()`), the conversation ends.

WHERE — the most-used SQL keyword after SELECT.

SELECT returns every row. WHERE returns only the rows that match. It's the SQL equivalent of pandas' **boolean mask** — but it runs in the database, not in Python.

```
where_patterns.py PYTHON

import sqlite3
conn = sqlite3.connect("students.db")
cursor = conn.cursor()

# 1. Single condition
cursor.execute("SELECT * FROM students WHERE age > 20")

# 2. Multiple conditions (AND / OR)
cursor.execute("""
    SELECT * FROM students
    WHERE age > 20 AND city = 'Amritsar'
""")

# 3. Pattern matching (LIKE)
cursor.execute("SELECT * FROM students WHERE name LIKE 'A%')
# → names starting with 'A'

# 4. IN a list
cursor.execute("SELECT * FROM students WHERE city IN ('Amritsar','Jalandhar')")

# 5. With parameters (ALWAYS use ? — never string formatting)
cursor.execute("SELECT * FROM students WHERE age > ? AND city = ?", (20, "Amritsar"))
rows = cursor.fetchall()
conn.close()
```

ALWAYS Use `?` for values — never f-strings or string concatenation. This prevents SQL injection, a real security issue.

PANDAS VS SQL SAME IDEA

Filter by age > 20

PD `df[df["age"] > 20]`

SQL `SELECT * FROM students WHERE age > 20`

Filter by city

PD `df[df["city"] == "Amritsar"]`

SQL `SELECT * FROM students WHERE city = 'Amritsar'`

SAME IDEA, DIFFERENT SYNTAX. Pandas runs in Python (**loads all data**). SQL runs in the database (**uses indexes, fast**).

CREATE TABLE — define your columns, types, and rules.

Before you can **INSERT** or **SELECT**, you need a table. CREATE TABLE defines the columns, their types, and the rules (constraints) every row must follow. **Run it once** — it sets the structure forever.

create_table.sql

SQL

```
-- The full CREATE TABLE statement
CREATE TABLE IF NOT EXISTS expenses (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  amount REAL NOT NULL,
  category TEXT NOT NULL DEFAULT 'Other',
  date TEXT NOT NULL,
  note TEXT,
  created_at TEXT DEFAULT (datetime('now'))
);

-- IF NOT EXISTS   → safe to run multiple times
-- PRIMARY KEY     → unique ID, auto-indexed
-- AUTOINCREMENT   → SQLite assigns 1, 2, 3...
-- NOT NULL        → required field, can't be empty
-- DEFAULT 'Other' → if not specified, uses 'Other'
-- DEFAULT (datetime('now')) → auto-fills timestamp
```

THE PARTS OF CREATE TABLE 6 PARTS

- CREATE TABLE** The keyword. Tells SQLite you're making a new table.
- IF NOT EXISTS** Safety. Won't error if the table already exists.
- name + type** `amount REAL`, `category TEXT`, `id INTEGER`. Like Day 6's dtypes.
- PRIMARY KEY** The unique ID column. One per table. Usually `id`.
- AUTOINCREMENT** SQLite assigns the next number automatically. You don't pass `id` in INSERT.
- Constraints** NOT NULL (required), DEFAULT (fallback), UNIQUE (no dups). The database enforces these.

- 1 IF NOT EXISTS** — Safe to run multiple times. If the table already exists, SQLite skips it. Always use this.
- 2 Constraints (NOT NULL, DEFAULT, UNIQUE)** — Rules the database enforces. NOT NULL = required. DEFAULT = fills if missing. UNIQUE = no duplicates. The database catches bad data at INSERT, not at runtime.
- 3 Cross-reference: Day 7** — Like defining columns in a pandas DataFrame, but here the types and rules are enforced by the database, not just by convention.

RUN IT ONCE CREATE TABLE defines the structure. You run it once when setting up the app. After that, you INSERT, SELECT, UPDATE, DELETE — but the **table stays**.

FOREIGN KEY — how tables talk to each other.

A database with one table is just a spreadsheet. The power of databases comes from **RELATIONSHIPS** — linking a student to their expenses, an order to its items, a user to their posts. Foreign keys are how you do it.

```
relationships.sql

-- Table 1: students (the "parent" table)
CREATE TABLE IF NOT EXISTS students (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  city TEXT
);

-- Table 2: expenses (the "child" table)
CREATE TABLE IF NOT EXISTS expenses (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  amount REAL NOT NULL,
  category TEXT,
  date TEXT,
  student_id INTEGER, -- the link!
  FOREIGN KEY (student_id) REFERENCES students(id)
);

-- CRITICAL: SQLite ignores FK constraints by default!
-- You MUST enable enforcement per connection:
PRAGMA foreign_keys = ON;

-- Without PRAGMA, student_id=999 silently
-- succeeds even if student 999 doesn't exist.
```

- 1 **student_id INTEGER** — A regular column that stores the ID of a student. This is the link.
- 2 **FOREIGN KEY (student_id) REFERENCES students(id)** — Tells SQLite: “every value in `student_id` must exist in `students.id`”. Prevents orphaned expenses. **But you MUST run `PRAGMA foreign_keys = ON` per connection** — SQLite ignores FK constraints by default!

THE RELATIONSHIP 1 → MANY

students		
id	name	city
1	Aarav	Amritsar
2	Priya	Jalandhar
3	Rahul	Amritsar

parent table

LINKS

expenses			
id	amt	category	stu_id
1	450	Food	1
2	120	Transport	1
3	680	Books	2
4	300	Food	3

child · references parent

● PRIMARY KEY in students ○ FOREIGN KEY in expenses — must match an existing id

CROSS-REF: DAY 6

Like pandas' `merge` / `join`. But here the relationship is **ENFORCED** by the database. You can't insert an expense for `student_id=999` if student 999 doesn't exist — the database won't let you.

JOIN — see data from two tables at once.

Foreign keys link tables. **JOIN** is how you USE that link in a query. Instead of two separate tables, you get one combined view — student name + their expense amount in the same row.

```
join_demo.py PYTHON

import sqlite3
conn = sqlite3.connect("app.db")
cursor = conn.cursor()

# INNER JOIN — only rows that match in both tables
cursor.execute("""
    SELECT students.name, expenses.amount, expenses.category
    FROM students
    INNER JOIN expenses ON students.id = expenses.student_id
""")

# Result: each row has the student's name
# AND their expense — combined from two tables
for row in cursor.fetchall():
    print(row)
# ('Aarav', 450, 'Food')
# ('Aarav', 120, 'Transport')
# ('Priya', 680, 'Books')

conn.close()

# LEFT JOIN — all students, even those with no expenses
# SELECT students.name, expenses.amount
# FROM students
# LEFT JOIN expenses ON students.id = expenses.student_id
# → includes students with NULL amount (no expenses yet)
```

- 1 **INNER JOIN** — Only matching rows. If a student has no expenses, they don't appear. Most common JOIN type.
- 2 **LEFT JOIN** — All rows from the left table (students), plus matching rows from the right (expenses). Students with no expenses show NULL for amount.
- 3 **ON students.id = expenses.student_id** — The condition that links the tables. This is the foreign key relationship from the last slide.

BEFORE VS AFTER JOIN 2x → 1x

⚡ BEFORE — TWO SEPARATE QUERIES

```
SELECT * FROM students;
→ (1, 'Aarav', 'Amritsar')
→ (2, 'Priya', 'Jalandhar')

SELECT * FROM expenses;
→ (1, 450, 'Food', 1)
→ (2, 120, 'Transport', 1)
```

🔗 AFTER — ONE JOIN QUERY

```
SELECT students.name, expenses.amount
FROM students
JOIN expenses
ON students.id = expenses.student_id;

→ ('Aarav', 450)
→ ('Aarav', 120)
→ ('Priya', 680)
```

CROSS-REF: DAY 6

Like pandas' `pd.merge(df1, df2, on='id')`. Same concept, different syntax. SQL JOIN is faster because it runs in the database, **not in Python memory**.

Aggregates — summarize a column in one number.

You used **SUM** in the exercise solution without learning it formally. Here's the full set. These functions take a whole column and return one value — the count, total, average, min, or max.

aggregates.py

PYTHON

```
import sqlite3
conn = sqlite3.connect("app.db")
cursor = conn.cursor()

# COUNT — how many rows?
cursor.execute("SELECT COUNT(*) FROM expenses")
# → (15,) — 15 expenses total

# SUM — total of a column
cursor.execute("SELECT SUM(amount) FROM expenses")
# → (12450.0,) — total spent

# AVG — average
cursor.execute("SELECT AVG(amount) FROM expenses")
# → (830.0,) — average per expense

# MIN / MAX — smallest / largest
cursor.execute("SELECT MIN(amount), MAX(amount) FROM expenses")
# → (50, 2000) — cheapest and most expensive

# GROUP BY — aggregate per category
cursor.execute("""
    SELECT category, SUM(amount)
    FROM expenses
    GROUP BY category
""")
# → [('Food', 4200), ('Transport', 3100), ...]

conn.close()
```

- 1

Aggregates return one row — `COUNT` / `SUM` / `AVG` / `MIN` / `MAX` take many rows and return one value. Unlike `SELECT`, which returns all matching rows.
- 2

`GROUP BY` changes everything — Without `GROUP BY`, you get one total. With `GROUP BY category`, you get one total PER category. The most useful pattern for dashboards.

PANDAS VS SQL

SAME MATH

OPERATION	PANDAS (DAY 6)	SQL (DAY 9)
Count rows	<code>len(df)</code>	<code>SELECT COUNT(*) FROM expenses</code>
Sum	<code>df["amount"].sum()</code>	<code>SELECT SUM(amount) FROM expenses</code>
Average	<code>df["amount"].mean()</code>	<code>SELECT AVG(amount) FROM expenses</code>
Min / Max	<code>df["amount"].min()</code>	<code>SELECT MIN(amount) FROM expenses</code>
Group by	<code>df.groupby("category")["amount"].sum()</code>	<code>SELECT category, SUM(amount) FROM expenses GROUP BY category</code>

SAME MATH, DIFFERENT ENGINE

pandas runs in Python (loads all data into memory). SQL runs in the database (uses indexes, only reads what it needs). For 28 rows, both are instant. For 28 million rows, SQL wins.

Three patterns you'll use in every dashboard.

`DISTINCT` gets unique values (for selectbox options). `ORDER BY + LIMIT` shows the top N rows. `IS NULL` finds empty fields. **All three appear in the exercise solution** — here's the formal version.

01 DISTINCT

Unique values for filter dropdowns

Build selectbox options from unique values. ↩ DAY 6 used

`df['category'].unique().tolist()` — this is the **SQL way**.

distinct_categories.sql

SQL

```
-- Get unique categories (for a selectbox)
SELECT DISTINCT category FROM expenses;
-- → ['Food', 'Transport', 'Books', 'Other']

-- Cross-ref: Day 6 — same as df["category"].unique()
-- Use this to build sidebar filter options
```

02 ORDER BY + LIMIT

Top-N and most-recent-N rows

Show the top N or most recent N rows. **Used in the exercise solution** to show the latest expenses.

recent_expenses.sql

SQL

```
-- Show the 5 most recent expenses
SELECT * FROM expenses
ORDER BY date DESC
LIMIT 5;

-- DESC = descending (newest first)
-- ASC = ascending (oldest first)
```

03 IS NULL

Handle optional / empty fields

The expense tracker's 'note' field can be empty — `IS NULL` finds those rows. ↩ DAY 5
session_state `None` check.

optional_notes.sql

SQL

```
-- Find expenses with no note (optional field left empty)
SELECT * FROM expenses WHERE note IS NULL;

-- Find expenses WITH a note
SELECT * FROM expenses WHERE note IS NOT NULL;

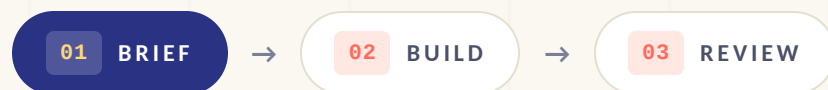
-- NULL is NOT the same as '' (empty string) or 0
```

THE FULL TOOLKIT

These three + `GROUP BY` (slide 19) = every dashboard query you'll ever write. Filter (`WHERE`), sort (`ORDER BY`), limit (`LIMIT`), unique (`DISTINCT`), summarize (`GROUP BY`), handle empties (`IS NULL`).

5 queries. 15 minutes. One pre-built **database**.

You've learned **SELECT, WHERE, DISTINCT, ORDER BY, LIMIT**, and aggregates. Now use them. I've pre-built an **expenses** database with 20 rows — your job is to answer 5 questions with SQL.



2 MIN • 15 MIN • 3 MIN

DATABASE

expenses.db

SQLite • single file • in the shared folder

ROWS

20 rows

Real expense records — amounts, categories, dates

QUERIES

5 SELECT

One per question. Each prints its result.

TOOLS

Python & sqlite3

Or DB Browser for SQLite if you prefer a GUI.

Answer 5 questions with SQL. Using a pre-built database.

I've created `expenses.db` with 20 expense rows. Download it from the shared folder. Open it in Python (or DB Browser). Write 5 queries — one per question below.

TASK

THE BRIEF

Download `expenses.db` from the shared folder. Write a Python script that connects to it and answers these 5 questions using SQL queries.

REQUIREMENTS

- ✓ Connect to `expenses.db` using `sqlite3`
- ✓ Write **5 SELECT queries** — one per question
- ✓ Print the results of each query
- ✓ Use **WHERE, DISTINCT, ORDER BY, LIMIT, and SUM** (one each)

BY THE END Every student has run **5 SQL queries** against a real database and seen the results.

THE 5 QUESTIONS

PATTERN → PROMPT

1 Show all expenses

```
SELECT * FROM expenses
```

→ How many rows are there?

2 Show only Food expenses

```
WHERE category = 'Food'
```

→ How many food expenses?

3 List all unique categories

```
SELECT DISTINCT category
```

→ What categories exist?

4 Show the 5 most recent expenses

```
ORDER BY date DESC LIMIT 5
```

→ What was the most recent expense?

5 What's the total amount spent?

```
SELECT SUM (amount)
```

→ What's the grand total?

RUN IT `python queries.py` · Each query prints its result. If you get stuck, check [slide 14](#) (cursor) and [slide 15](#) (WHERE clause).

Today you learned what a database **IS**. Tomorrow you **build** one.

JSON, database fundamentals, SQL queries, **CREATE TABLE**, **FOREIGN KEY**, **JOIN**, aggregates — that's the foundation. Tomorrow we take it into Streamlit and build the expense tracker with real persistence.

ASK NOW



Open the floor

- ✓ Which SQL command felt most natural — SELECT, WHERE, or JOIN?
- ✓ Did the **cursor** concept click, or does it feel weird?
- ✓ Anything from the **exercise** that didn't work?

GET UNSTUCK



Between sessions

- ✓ **Email** • hello@zlaark.com
For SQL questions, paste your query + the error.
- ✓ **Web** • www.zlaark.com
Program updates and resources.
- ✓ **Practice** — re-run today's 5 queries with different WHERE clauses.

BUILDING WITH SQLITE



What's next • Day 10

- ✓ **SQLite deep-dive** — the full connect → CREATE → INSERT → commit pattern
- ✓ **SQLite in Streamlit** — the helper-function pattern
- ✓ **Exercise:** rebuild Day 8's expense tracker with a real database